Aufgabe: Tamagotchi-Simulator – Objekte, Interfaces & Exceptions

Was ist ein Tamagotchi?

Ein Tamagotchi ist ein kleines virtuelles Haustier, das auf einem digitalen Gerät lebt. Es hat eigene Bedürfnisse wie Hunger, Freude/Happiness und Schlaf/Sleepiness.

- Pflege: Du musst dein Tamagotchi regelmäßig füttern, spielen und schlafen legen, sonst wird es unglücklich oder krank.
- Interaktion: Du kannst es umarmen, füttern, mit ihm spielen oder es schlafen legen.
- Statusanzeigen: Hunger, Happiness und Sleepiness werden meist über Balken oder Zahlen angezeigt.

Das Ziel ist, das Tamagotchi gesund und glücklich zu halten.
In diesem Projekt simulieren wir genau dieses Verhalten in einer grafischen JavaFX-Oberfläche, sodass ihr ein eigenes virtuelles Haustier programmieren könnt.

Aufgabe im Detail

Du arbeitest in einem Unternehmen, das Spiele entwickelt. Es ist ein recht frisches Unternehmen und versucht sich gerade an leichten Spielen aus. In dem Unternehmen ist bereits ein Softwareentwickler eingestellt, der für die grafische Oberfläche zuständig war. Diese steht somit schon.

Er hat sich aber komplett überarbeitet, weil er noch nie was mit JavaFx gemacht hatte und sich da ganz viel einarbeiten und ausprobieren musste. Da der Softwareentwickler aber wusste, was das Programm braucht, um zu arbeiten, hat er eine grobe Struktur mit Hilfe von abstrakten Klassen vorformuliert. Nun kommst du also ins Spiel.
Da bekannt ist, dass du dich mit abstrakten Klassen und Interfaces gut auskennst, setzt dich das Unternehmen für den zweiten Schritt der Umsetzung ein.

Arbeitsauftrag:

1. Lasse den Code laufen. Drücke auf die Buttons und DropDown-Menüs. Was fällt dir auf?

2. Lese dir den JavaDoc für die einzelnen Klassen und Methoden durch.

3. Such nach möglichen TODO – Kommentaren im Code, die vom Vorgänger hinterlassen wurden. Verschaffe dir somit einen Überblick über den Code.

4. Versuche das Programm zum Laufen zu bekommen, indem du die ToDos abarbeitest. Zur Orientierung kannst du die Stichpunkte weiter unten verwenden.

5. Bonus: Weiter unten stehen weitere Möglichkeiten, die man dem Spiel hinzufügen kann. Wer Lust und Zeit hat, darf diese gerne umsetzen. Gerne stellt der Team-Lead (Dozentin :D) auch eine Hilfestellung zur Verfügung, damit **jeder** diese Aufgaben lösen kann. Er muss nur angesprochen werden.

- Food, Pets, Toys:
  - Implementiere mindestens eine weitere abstrakte Klasse, die von einer der bestehenden abstrakten Klassen erbt.
  - Jede abstrakte Klasse soll die toString()-Methode überschreiben, sodass sie nicht mehr den HashCode des Objektes zurückgibt, sondern den Namen.
    Soll man das in der abstrakten Klasse oder in den Subklassen lösen?
- Interfaces:
  - Erzeuge das Interface Huggable mit einer Methode hug
  - Dieses soll von Pet implementiert werden.
  - Einige Tiere mögen es umarmt zu werden, während andere das gar nicht ab haben können 😊 Heißt, je nachdem welches Tier, soll bei einer Umarmung die Happiness hochgehen oder abstürzen :D.
  - Füge JavaDoc hinzu.

Optional: Statt den Exceptions in der Konsole soll in der UI angezeigt werden, was das Problem ist.

---

2. UIHelper

- In utils/UIHelper.java existieren die Methoden:
    - populateFoodDropDown(List<Food> items)
    - populateToyDropDown(List<Toy> items)
    - populatePetDropDown(List<Pet> items)
- Aufgabe:
    - Füge der Combobox die Liste der erzeugten Tiere/Futter/Spielzeuge hinzu Nutze dafür: controller.get...ComboBox().getItems().addAll(list);
    - Optional kannst du auch die .add() Methode nutzen. Dann musst du jedoch vorher nochmal mit einer for-Schleife die einzelnen Elemente aus der Liste ziehen.

---

3. OPTIONAL, jedoch machbar für jeden 😊:

Folgende Aufgaben können gelöst werden, wenn man sich im ViewController etwas umschaut und den Code versteht.

- Dynamische Mood-Logik. Dafür ist der TamagotchiViewController wichtig. Explizit die updateMoodLabel() – Methode.
    - 😴 → müde
    - 😐 → neutral
    - 😊 → glücklich
    - 💀 → tot

    - 😟 → traurig
    - 🙄 → gelangweilt bei zu viel Spielen
    - 💤 → schläft
- Animations- und Farbwechsel. Dafür muss in der TamagotchiViewController-Klasse eine weitere Methode updateColorOfPet() implementiert werden, welcher die verschiedenen Gemütszustände des Tiers abfragt und dementsprechend die Farbe anpasst. Nutzt dafür die Variable colorOfPet und die setColor-Methode.
    - Pet-Farbe ändert sich je nach Stimmung
    - Blink-Effekt bei Freude
    - Rot bei Hunger
    - Transparent/blau beim Schlafen
    - Circle „hüpft“ beim Spielen
- Overplaying / Overfeeding. Es sollten zwei weitere Methoden programmiert werden. Einmal checkIfOverplayed() und checkIfOverfed(). Je nachdem wie oft man beispielsweise mit dem gleichen Spielzeug gespielt hat oder wie oft man dem Tier essen gibt, obwohl er schon so voll ist, soll sich die Laune (moodLabel) ändern. Überlege dir, wo du deine Methoden einsetzen musst, damit deine Logik funktioniert.
    - Übermäßiges Spielen → Happiness sinkt, MoodLabel zeigt 🙄
    - Übermäßiges Füttern → Hunger-Balken kann nicht über max hinaus, MoodLabel zeigt 😐 oder 😟
- Food/Toy-Nebenwirkungen:
    - Steak → Hunger stark hoch, Happiness leicht erhöht
    - Puzzle → Happiness stark hoch, Sleepiness leicht reduziert
- Kombinationsbonus:
    - Füttern + direkt danach Spielen → Happiness steigt extra
    - Zu viel gleichzeitiges Füttern + Spielen → Mood sinkt
- Für besonders schnelle und bereits Erfahrene:
    - Level hinzufügen
    - Hinzufügen, dass man über die UI Tiere, Essen und Spielzeuge hinzufügen kann. (Vielleicht auch nur durch „Kaufoptionen“)